Block Diagrams & Cross-Domain Interfaces

Visual languages for systemic design

SEAtS | 211939

University of Southampton

Intended Learning Outcomes

By the end of this lecture, you will be able to:

1. Explain how block diagrams model information, energy, and material flows within complex systems.
2. Construct multi-level block diagrams showing system decomposition and cross-domain interfaces.
3. Integrate feedback and control loops within and between domains (hardware, software, learning).
4. Apply systems thinking to identify boundaries, stakeholders, and interactions in multi-component systems.
5. Evaluate how distributed optimisation and meta-modelling principles support system-level design consistency.

© University of Southampton

Agenda

- Block Diagrams as System Language
- Systems Thinking Foundations
- Functional Decomposition
- Anatomy of a Block Diagram
- Documenting System Views: The 4+1 Model
- Cross-Domain Interfaces
- Interface Characterisation
- Boundaries & Stakeholders
- Feedback Across Domains
- Case Study: Distributed Robotic Arm
- Distributed Optimisation Patterns
- Learning & Adaptation

© University of Southampton

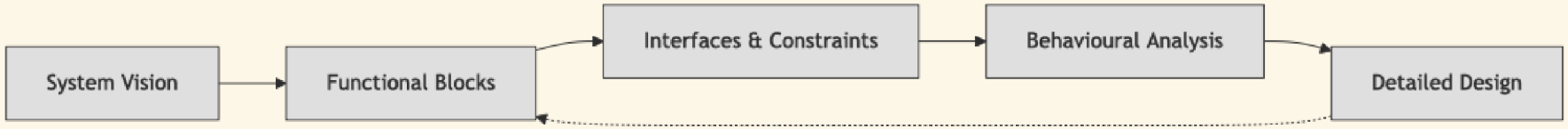
Block Diagrams as System Language

A block diagram is more than a schematic. It is a language for abstracting physical complexity into functional relationships—helping teams reason about behaviour, communication, and dependencies before committing to detailed design.

Practice spotlight

- Distributed optimisation remains stable only when subsystem interfaces are explicit, letting teams coordinate performance without constant reintegration.
- Visual frameworks act as shared “sense-making artefacts” across stakeholders, turning block diagrams into boundary objects for engineers, designers, and managers.

© University of Southampton



Block diagrams sit between vision and detailed design, enabling iterative refinement of interfaces and behaviour.

University of Southampton
© University of Southampton



© University of Southampton

Systems Thinking Foundations

Systems thinking treats phenomena as interdependent, dynamic, and adaptive. Effective block diagrams externalise key principles:

✓ Holism

- Whole-system behaviour emerges from part interactions.
- Diagrams expose relationships that cannot be seen by isolating components.

ⓘ Interdependence

- Changes propagate through feedback and information flow.
- Visualising loops uncovers potential ripple effects.

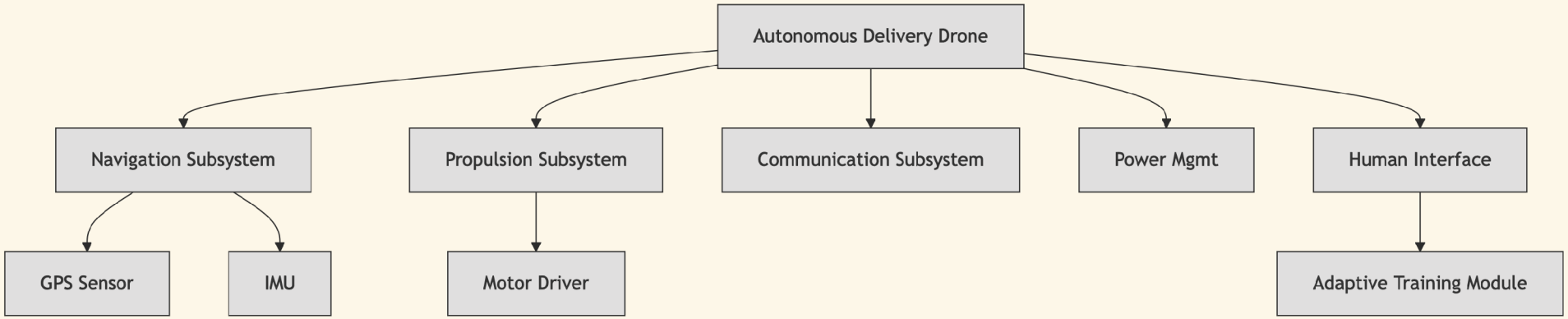
⚠ Hierarchy

- Systems nest into subsystems and components.
- Multi-level diagrams organise complexity across scales.

Example: Drone stability requires software (flight algorithms), hardware (sensors, actuators), and learning modules (operator training). Only the connections reveal emergent stability.

© University of Southampton

Hierarchical block diagrams clarify decomposition from system to component level.



© University of Southampton



© University of Southampton

Functional Decomposition

Designers break complex systems into manageable subsystems—mirroring both technical and organisational boundaries.

✓ Hierarchical structure

1. **System level** (e.g., Autonomous Delivery Drone)
2. **Subsystems** (Navigation, Propulsion, Communication, Power, Human Interface)
3. **Components** (GPS sensor, motor driver, neural controller)

⚠ Interface discipline

- Clearly defined interface variables (mass, stiffness, control signals) let subsystems optimise locally while staying aligned to system goals.
- Block diagrams provide the coordination plan linking distributed teams.

© University of Southampton

Anatomy of a Block Diagram

Block essentials

- **Inputs**: quantities entering the subsystem.
- **Outputs**: signals or resources leaving it.
- **Internal function**: transformation, computation, or process.

Connection semantics

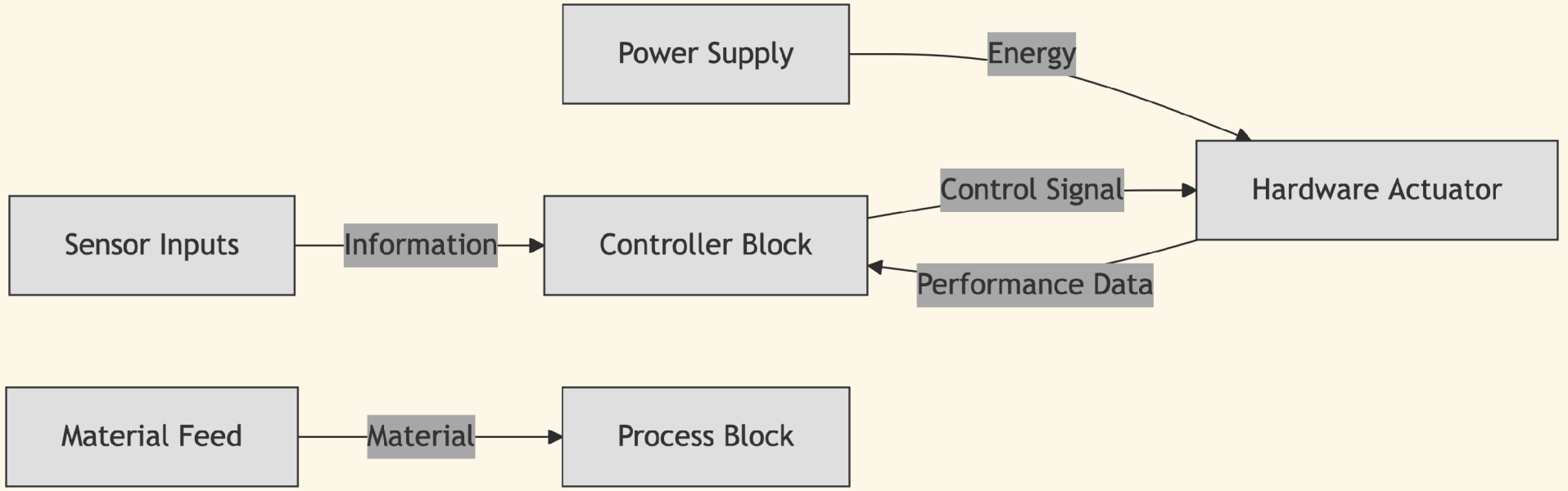
- Information flow: data, commands, telemetry.
- Energy flow: power, force, voltage.
- Material flow: mass, fluids, physical items.

Feedback loops

- Regulate performance by feeding outputs back into inputs.
- Reveal control structures and stability considerations.

© University of Southampton

Use line labels or styles to differentiate information, energy, and material flows.



© University of Southampton



© University of Southampton

Documenting System Views: The 4+1 Model

The block diagrams we've drawn so far all mix levels of abstraction. Kruchten's **4+1 view model** separates them: each view answers one question about the system, and is best documented with a specific family of UML (or UML-like) diagrams.

```
graph TD
    SA[System Architecture]
    SA --> LV["Logical View<br/>What functions?"]
    SA --> PRV["Process View<br/>How it executes?"]
    SA --> DV["Development View<br/>How it evolves?"]
    SA --> PV["Physical View<br/>Where it runs?"]

    LV --> LVD[Class / Object diagrams]
    PRV --> PRVD[Sequence / Activity diagrams]
    DV --> DVD[Component / Package diagrams]
    PV --> PVD[Deployment diagrams]

    SV(["Scenarios +1<br/>Does it hang together?"]) --> SVD[Use case diagrams]
    SV -.-> LV
    SV -.-> PRV
    SV -.-> DV
    SV -.-> PV
```


© University of Southampton

A Diagram Type for Each View

Each view is documented with the UML (or UML-like) diagram best suited to the question it answers — Kruchten's "+1" is the **Scenarios view**: a handful of use cases that cut across all four, used to check they're actually consistent with each other.

View	Answers	Typical diagram(s)
Logical	What functions?	Class diagrams, communication/object diagrams
Process	How it executes?	Sequence diagrams, activity diagrams (concurrency, timing)
Development	How it evolves?	Component diagrams, package diagrams
Physical	Where it runs?	Deployment diagrams
Scenarios (+1)	Does it all hang together?	Use case diagrams

© University of Southampton

Logical View — Class & Object Diagrams

- **Answers**: what functions and responsibilities exist, and how they relate.
- Captures modules, their data, and their associations — independent of runtime behaviour or deployment.
- **Class diagrams** show static structure; **object/communication diagrams** show a snapshot of specific instances collaborating.

```
classDiagram
    class FlightController
    class NavigationModule
    class PropulsionModule
    class CommunicationModule
    class PowerModule
    FlightController --> NavigationModule : requests position
    FlightController --> PropulsionModule : sends thrust command
    FlightController --> CommunicationModule : sends telemetry
    FlightController --> PowerModule : reads battery state
```


© University of Southampton

Process View — Sequence & Activity Diagrams

- **Answers**: how the system executes — concurrency, timing, synchronisation between runtime processes.
- **Sequence diagrams** order the messages exchanged between participants; **activity diagrams** show branching/parallel control flow.
- Same modules as the Logical View, now shown interacting over time.

```
sequenceDiagram
    participant GCS as Ground Control
    participant Nav as Navigation Module
    participant FC as Flight Controller
    participant Prop as Propulsion Module
    GCS->>Nav: New waypoint
    Nav->>FC: Target heading & altitude
    FC->>Prop: Motor thrust commands
    Prop-->>FC: Motor status
    FC-->>GCS: Telemetry update
```


© University of Southampton

Development View — Component & Package Diagrams

- **Answers**: how the system evolves — the programmer's-eye view of modules, libraries, and build dependencies.
- **Component diagrams** expose interfaces between deployable units; **package diagrams** group related code and show compile/build-time dependencies.
- Organised by repository, not by runtime behaviour.

```
graph TD
    subgraph Repo1["repo: flight-firmware"]
        FCPkg[Flight Controller]
        NavPkg[Navigation Algorithms]
    end
    subgraph Repo2["repo: ground-station"]
        UIPkg[Operator UI]
        TelemetryPkg[Telemetry Client]
    end
    subgraph Repo3["repo: motor-drivers"]
        DriverPkg[Motor Driver Library]
    end
    UIPkg --> TelemetryPkg --> FCPkg
    FCPkg --> NavPkg
    FCPkg --> DriverPkg
```


© University of Southampton

Physical View — Deployment Diagrams

- **Answers**: where the system runs — software mapped onto hardware, nodes, and network topology.
- **Deployment diagrams** show processors, devices, and the protocols connecting them.
- Surfaces latency, bandwidth, and failure-domain concerns the other views hide.

```
graph TB
    subgraph GroundStation["Ground Station"]
        Laptop["Operator Laptop"]
        RadioGS["Ground Radio"]
    end
    subgraph Drone["Drone Airframe"]
        FCBoard["Flight Controller Board"]
        GPSModule["GPS Receiver"]
        ESC["Motor ESCs"]
        RadioAir["Onboard Radio"]
    end
    Laptop --> RadioGS
    RadioGS -- "915MHz telemetry link" --> RadioAir
    RadioAir --> FCBoard
    FCBoard --> GPSModule
    FCBoard --> ESC
```


© University of Southampton

Scenarios (+1) View — Use Case Diagrams

- **Answers**: does it all hang together? Not a separate structure — a handful of significant use cases that exercise the other four views at once.
- **Use case diagrams** show actors and the goals they want the system to achieve, linked by include/extend relationships.
- Used to sanity-check that the Logical, Process, Development, and Physical views are consistent with each other.

```
flowchart LR
    Operator((Operator))
    Regulator((Regulator))
    Drone((Drone))
    Operator --- UC1([Plan delivery route])
    Operator --- UC2([Monitor battery & abort mission])
    Regulator --- UC3([Audit flight log])
    Drone --- UC4([Avoid no-fly zone])
    UC1 -.includes.-> UC5([Check geofence])
    UC4 -.includes.-> UC5
```


© University of Southampton

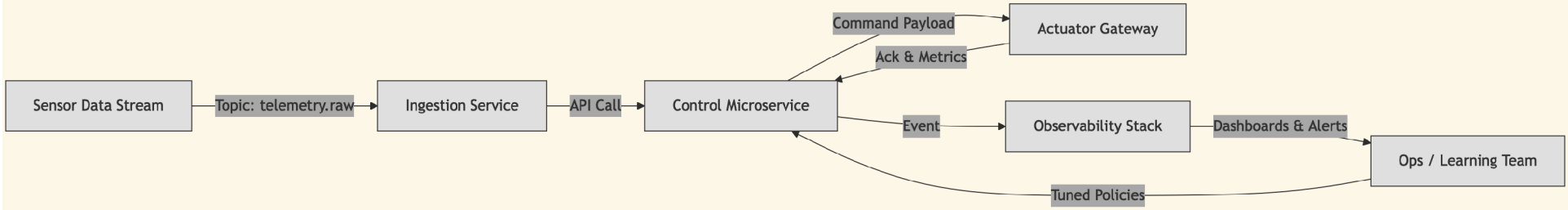
Software-Oriented Control Flow

Shift the lens

- Map abstract services and data contracts instead of physical signals.
- Highlight orchestration, data governance, and monitoring responsibilities.

© University of Southampton

Reframing the same loop in software terms exposes integrations, protocols, and observability hooks.



© University of Southampton



© University of Southampton

Cross-Domain Interfaces

Modern systems combine hardware, software, and learning (human or AI). Interfaces span physical signals, data protocols, and cognitive workflows.

Domain	Typical functions	Interface examples
Hardware	Sensing, actuation, power conversion	Analog/digital signals, mechanical couplings
Software	Data processing, decision-making, control logic	APIs, middleware, message buses
Learning (Human/AI)	Adaptation, perception, feedback interpretation	Dashboards, reward signals, UX interactions

Example: Autonomous vehicle system

- Hardware block: LIDAR, cameras, actuators
- Software block: Perception, planning, control
- Learning block: Driver model or reinforcement learning agent

© University of Southampton

Cross-domain pathways highlight responsibilities and integration bottlenecks.



© University of Southampton



© University of Southampton

Interface Characterisation

Interfaces fall naturally into complementary categories:

✓ Functional

- Defines what one block provides to another (e.g., torque command → motor).
- Focus on capability and performance parameters.

ⓘ Physical

- Mechanical, electrical, or material couplings.
- Consider tolerances, connectors, and environmental constraints.

⚠ Informational

- Data formats, timing, synchronisation.
- Includes security, privacy, and protocol standards.

In multi-domain systems, a single signal (e.g., temperature reading) may embody all three interface types.

© University of Southampton

Boundaries & Stakeholders

System boundaries are technical and organisational. Stakeholder mapping ensures interfaces align with responsibilities and ethics.

Boundary questions

- Who supplies each subsystem?
- What data crosses organisational lines?
- How are ethics, privacy, and security enforced at interfaces?

Example: Smart university lab

- Learning analytics software depends on lab sensors and student feedback.
- Block diagram clarifies data ownership, consent flows, and accountability across departments.

© University of Southampton

Feedback Across Domains

Feedback is the lifeblood of dynamic systems. Cross-domain block diagrams often contain nested loops:

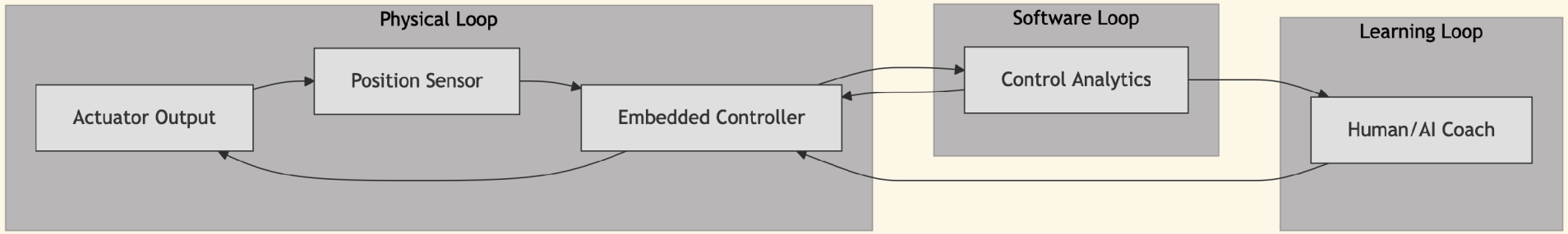
- **Hardware feedback**: physical control (speed sensors, torque reports).
- **Software feedback**: algorithm tuning (adaptive control, anomaly detection).
- **Learning feedback**: human/AI adaptation (dashboards, reinforcement signals).

⚠ Multi-layer interactions

- Inner loops stabilise physical behaviour.
- Outer loops adapt strategy or policy.
- Poorly aligned loops can conflict—plan update rates and priorities explicitly.

© University of Southampton

Nested loops reveal timing and authority relationships across domains.



© University of Southampton



© University of Southampton

Case Study: Distributed Robotic Arm

A lightweight robotic arm uses four modular joints with mechanical, control, and sensing subsystems.

✓ System goal

- Maintain <1mm displacement while minimising mass.

ⓘ Component goals

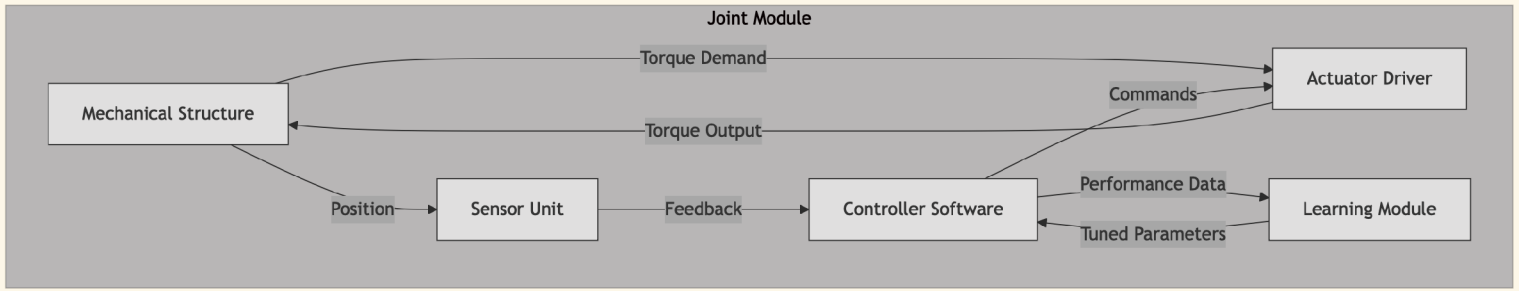
- Optimise stiffness, geometry, and control responsiveness per joint.

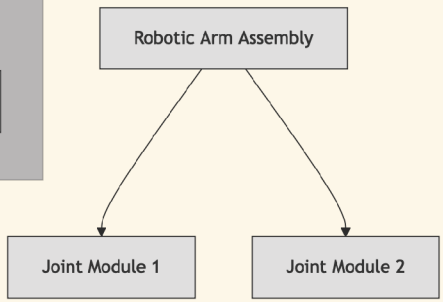
⚠ Interfaces

- Mechanical ↔ Electrical: torque and position signals.
- Electrical ↔ Software: digital control commands.
- Software ↔ Learning: reinforcement signals for optimisation.

© University of Southampton

Visualising subsystems and interfaces surfaces potential latency, feedback delay, or energy loss points.





© University of Southampton



© University of Southampton

Distributed Optimisation Patterns

Distributed optimisation replaces monolithic design when teams coordinate through shared constraints and interfaces:

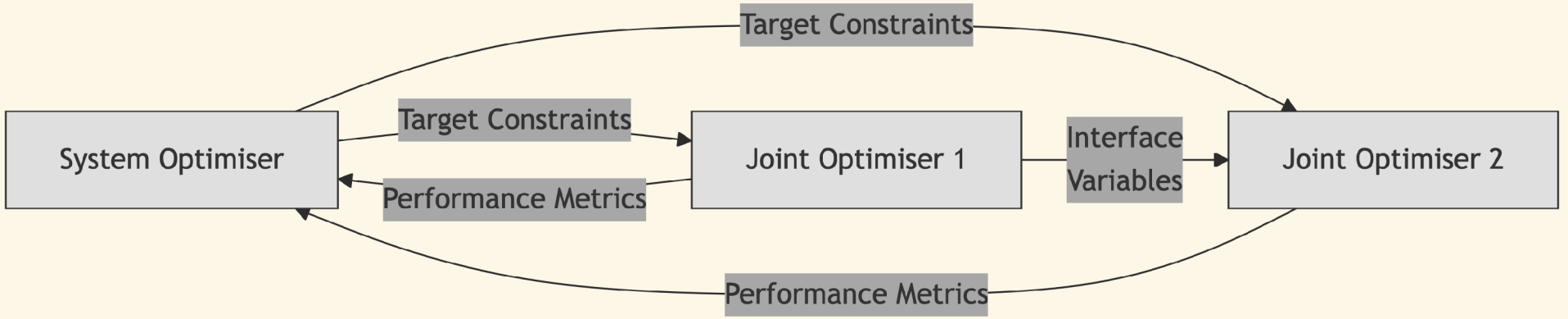
Workflow

1. System-level optimisation sets target stiffness/mass.
2. Component optimisations run autonomously to meet targets.
3. Interface variables (e.g., stiffness matrices) maintain consistency.

Block diagrams naturally describe this decomposition—linking a system optimiser to component optimisers via information and constraint flows.

© University of Southampton

Distributed optimisation relies on interface contracts rather than centralised control.



© University of Southampton



© University of Southampton

Learning & Adaptation

Contemporary systems (autonomous vehicles, smart grids, educational robotics) learn from experience. Block diagrams capture adaptive processes by adding learning feedback blocks:

✓ Learning loop

- **Input**: sensor data or human observations.
- **Process**: model training or reflective analysis.
- **Output**: updated control policies or improved human performance.

Embedding learning within systemic design supports resilience and sustainability—interfaces must tolerate module evolution.

© University of Southampton

Summary

- Block diagrams represent system structure, hierarchy, and feedback.
- They bridge hardware, software, and learning domains via explicit interfaces.
- Distributed and systemic design methods rely on interface clarity and meta-modelling.
- When combined with systems thinking, block diagrams become strategic tools for orchestrating complex, adaptive systems.

© University of Southampton